

PROJECT REPORT

Computational Modeling of Rocket Trajectory Dynamics with Multistage Separation and Atmospheric Drag

A Physics-Based Simulation Project

Developed after the completion of the course

Rocket Propulsion

Developed by

SURYAPRIYA C S
M.Sc. Space Science and Technology
St. Albert's College, Ernakulam

Date: 27 MAY 2026

TABLE OF CONTENTS

Abstract	
1 Introduction	4
1.1 Newton's Second Law of Motion	5
1.2 Gravitational Force	5
1.3 Atmospheric Drag	6
1.4 Rocket Thrust and Mass Flow	6
1.5 Multistage Rocket Principle	7
2 System Implementation	8
2.1 Programming Language	8
2.2 Libraries Used	8
3 Key Features	10
3.1 2D Motion Simulation	10
3.2 Atmospheric Model	10
3.3 Multistage Rocket System	11
3.4 AI Optimization	12
4 Simple Vertical Rocket Motion Model	13
4.1 Basic Assumptions	13
4.2 Newton's second Law for Vertical Motion	13
4.3 Mass Variation Due to Fuel Burn	14
4.4 Numerical Motion Update	14
4.5 Physical Interpretation	14
4.6 Vertical Rocket Motion Diagram	15
5 Rocket Visualization and Advanced Simulation Models	16
5.1 Rocket Visualization System	16
5.2 Realistic Rocket Physics Model	18

5.3 Rocket Motion Visualization and Graph Analysis	19
5.4 AI-Based 2D Rocket Optimization	20
5.5 Advanced 3D AI-Based Aerospace Simulation	22
6 Conclusion	24
7 References	25

ABSTRACT

This project presents a physics-based simulation of rocket flight developed using Python, designed to realistically model the motion and behavior of a rocket under the influence of fundamental physical principles. The simulation is built upon Newton's laws of motion, which govern how forces such as thrust, gravity, and atmospheric drag interact to determine the rocket's acceleration, velocity, and overall trajectory. By numerically solving these equations of motion, the system provides a step-by-step representation of how a rocket would behave in real-world conditions rather than relying on simplified or purely visual animations.

A key feature of the simulator is the inclusion of gravitational variation with altitude. Instead of assuming a constant gravitational field, the model accounts for the decrease in gravitational strength as the rocket moves farther from the Earth's surface. This enhances the physical realism of the simulation, especially for high-altitude and near-space flight conditions.

In addition, atmospheric drag is incorporated to simulate the resistance offered by air molecules during ascent. The drag force depends on factors such as velocity, air density, and the rocket's cross-sectional area, and it plays a crucial role in reducing speed at lower altitudes where the atmosphere is denser. As altitude increases and air density decreases, the effect of drag diminishes, allowing the rocket to accelerate more efficiently.

The simulator also includes a multistage rocket separation mechanism, which reflects real-world rocket design practices. In this system, different stages of the rocket detach once their fuel is exhausted, reducing the overall mass of the vehicle. This staged reduction in mass improves thrust efficiency and allows the remaining stages to achieve higher velocities and altitudes. The model dynamically updates the mass and propulsion parameters after each separation event.

Furthermore, the simulation extends into a two-dimensional motion framework, enabling the analysis of both vertical and horizontal components of rocket flight. This allows the trajectory of the rocket to be visualized in a plane rather than a single vertical line, making the model more realistic and suitable for studying flight paths, range, and directional control.

An initial AI-based optimization module is also introduced in the project. This module is designed to enhance flight efficiency by analyzing different flight parameters such as thrust, fuel consumption, and trajectory adjustments. Although in a preliminary stage, this component lays the foundation for intelligent optimization systems that can improve fuel usage and overall mission performance through adaptive parameter tuning.

Overall, the objective of this project is to develop a modular and extensible computational framework for rocket simulation. The system is designed in a way that allows future enhancements, such as more advanced aerodynamic models, orbital mechanics integration, and machine learning-based control systems. This makes the simulator not only a tool for understanding rocket physics but also a scalable platform for further research and development in computational aerospace modeling.

Keywords: Rocket Propulsion, Numerical Simulation, Atmospheric Drag, Multistage Rocket, Python, Computational Physics, Trajectory Dynamics

INTRODUCTION

Rocket propulsion systems are governed by the fundamental principles of classical mechanics, with Newton's Second Law of Motion serving as the primary framework for analyzing their dynamic behavior. Accurate modeling of rocket flight requires consideration of multiple time-dependent and environmental effects that significantly influence trajectory and performance.

One of the key factors in rocket dynamics is the continuous variation of mass due to fuel consumption. As propellant is burned, the total mass of the rocket decreases, resulting in a nonlinear variation in acceleration as described by Newtonian mechanics. This time-varying mass makes the system inherently dynamic and necessitates numerical simulation techniques for accurate analysis.

In addition, gravitational acceleration is not strictly constant and varies with altitude according to the inverse-square law. Although this variation is relatively small near Earth's surface, it becomes increasingly significant at higher altitudes and must be incorporated for realistic long-range or near-space trajectory modeling.

Atmospheric drag also plays a crucial role in determining rocket motion, particularly during the initial ascent phase. The drag force is dependent on velocity, atmospheric density, and the aerodynamic characteristics of the vehicle. As altitude increases and atmospheric density decreases, the effect of drag gradually diminishes, altering the overall acceleration profile of the rocket.

Modern rocket systems also employ multistage configurations to enhance efficiency. Stage separation reduces the overall mass of the vehicle after fuel depletion in each stage, thereby improving the thrust-to-mass ratio of the remaining structure. This staged propulsion strategy is essential for achieving high-altitude and orbital missions.

Furthermore, thrust generated by the propulsion system varies over time depending on engine performance and fuel consumption characteristics. Accurate modeling of thrust profiles is therefore necessary for realistic simulation of flight dynamics.

In this work, a computational framework is developed using Python to simulate rocket flight under realistic physical conditions. The proposed model integrates variable mass dynamics, gravitational variation, atmospheric drag, and multistage separation within a two-dimensional motion framework. The simulation enables visualization and analysis of rocket trajectories, providing a computational tool for studying rocket dynamics and performance optimization.

1 Theoretical Background

1.1 Newton's Second Law of Motion

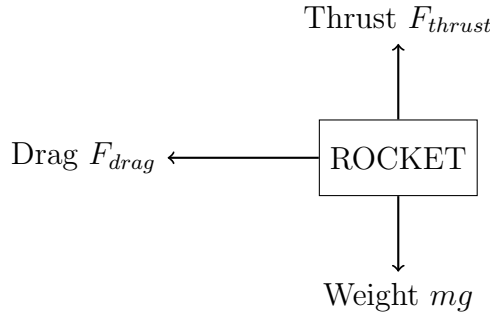
Newton's Second Law of Motion forms the fundamental basis for modeling rocket dynamics in this simulation. It states that the net force acting on an object is equal to the product of its mass and acceleration. In mathematical form, this is expressed as:

$$F = ma \quad (1)$$

In the case of a rocket, multiple forces act simultaneously during flight, and therefore the motion is governed by the net force acting on the system. The net force is obtained by combining the thrust produced by the engine, the gravitational force pulling the rocket downward, and the aerodynamic drag opposing the motion. Thus, the net force can be written as:

$$F_{net} = F_{thrust} - F_{drag} - F_{weight} \quad (2)$$

Since the rocket continuously burns fuel during flight, its mass decreases with time, making the system dynamically changing. As a result, even if thrust remains constant, the acceleration of the rocket increases as mass reduces. This time-dependent behavior makes rocket motion a non-linear physical system that requires numerical simulation for accurate analysis.



1.2 Gravitational Force

Gravitational force is the attractive force exerted by the Earth on the rocket, which acts in the downward direction toward the Earth's center. It is described by Newton's law of universal gravitation as:

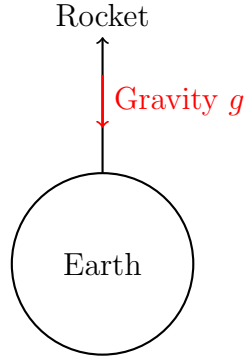
$$F_g = \frac{GMm}{r^2} \quad (3)$$

where G is the universal gravitational constant, M is the mass of the Earth, m is the mass of the rocket, and r is the distance from the center of the Earth to the rocket.

In terms of acceleration, gravity is represented as:

$$g = \frac{GM}{r^2} \quad (4)$$

Near the Earth's surface, this variation is small and gravity is often approximated as constant. However, in high-altitude or space-flight conditions, the reduction in gravitational strength becomes significant and must be considered. In this project, gravity is modeled as a variable parameter depending on altitude.



1.3 Atmospheric Drag

Atmospheric drag is the resistive force experienced by the rocket as it moves through the Earth's atmosphere. This force always acts opposite to the direction of motion and reduces velocity. The drag force is mathematically expressed as:

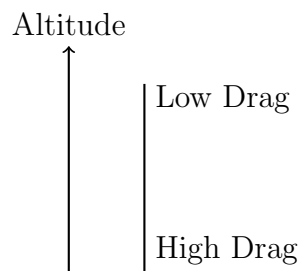
$$F_d = \frac{1}{2} C_d \rho A v^2 \quad (5)$$

where C_d is the drag coefficient, ρ is air density, A is cross-sectional area, and v is velocity.

Air density decreases exponentially with altitude:

$$\rho(h) = \rho_0 e^{-h/H} \quad (6)$$

This means drag is strongest near the Earth's surface and decreases at higher altitudes.



1.4 Rocket Thrust and Mass Flow

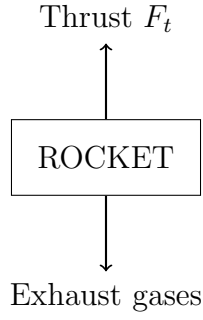
Thrust is the primary force responsible for lifting the rocket against gravity and atmospheric resistance. It is generated by the expulsion of exhaust gases from the engine. The thrust produced is given by:

$$F_t = \dot{m} v_e \quad (7)$$

where $\dot{m}$ is the mass flow rate and v_e is the exhaust velocity.

The mass of the rocket decreases over time due to fuel consumption:

$$m(t) = m_0 - \dot{m}t \quad (8)$$



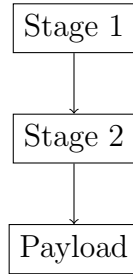
1.5 Multistage Rocket Principle

The multistage rocket principle is based on improving efficiency by reducing mass during flight. A rocket is divided into multiple stages, each containing its own fuel and engine system. Once a stage completes its fuel consumption, it is detached from the main vehicle, thereby reducing the overall mass.

The mass after stage separation is given by:

$$m_{k+1} = m_k - m_{stage,k} \quad (9)$$

This reduction increases thrust-to-weight ratio and improves overall performance, enabling higher velocity and altitude.



2 System Implementation

The system implementation of the rocket flight simulator is carried out using Python, which is a widely adopted programming language in scientific computing and computational physics. The choice of Python is motivated by its simplicity, readability, and strong ecosystem of libraries that support numerical modeling, simulation, and data visualization. The overall system is designed in a modular manner, where different components handle physics calculations, numerical updates, and graphical visualization independently, ensuring flexibility and scalability for future enhancements.

The simulator works by discretizing time into small steps and iteratively computing the motion of the rocket using physics-based equations. At each time step, forces such as thrust, gravity, and drag are calculated, and the resulting acceleration is used to update velocity and position. This numerical approach allows the system to simulate continuous motion using computational methods.

2.1 Programming Language

The entire simulation is implemented in Python, a high-level programming language extensively used in scientific computing, engineering, and data science applications. Python is preferred for this project because it allows efficient implementation of complex mathematical models with relatively simple syntax.

In the context of this rocket simulation, Python enables:

- Easy formulation of physics equations
- Real-time numerical computation of motion
- Flexible handling of vectors and arrays
- Rapid development and testing of simulation models

Additionally, Python supports integration with scientific libraries, which significantly enhances the performance and capability of the simulation system without requiring low-level programming.

2.2 Libraries Used

NumPy: NumPy is a core library used for numerical computation. It provides support for multi-dimensional arrays and efficient mathematical operations on large datasets. In this simulation, NumPy is used to handle vectors such as position, velocity, and acceleration, and to perform repeated calculations required in time-stepped motion updates.

Matplotlib: Matplotlib is used for visualization of the rocket's motion and trajectory. It enables graphical representation of altitude vs time, velocity vs time, and 2D flight paths. Visualization is essential for analyzing and validating simulation results.

Math Module: The built-in Python Math module is used for fundamental mathematical operations such as trigonometric functions, exponential functions, and power operations required in physics calculations.

SciPy (Optional / Future Enhancement): SciPy is considered for future improvements of the simulator. It provides advanced numerical integration and optimization tools that can improve the accuracy of the simulation.

3 Key Features

The proposed rocket flight simulation system is designed as a physics-driven computational framework that integrates multiple layers of real-world aerospace dynamics. The objective of this system is not only to simulate rocket motion but also to replicate the underlying physical environment in which real rockets operate. This includes motion in a two-dimensional plane, atmospheric variation with altitude, multistage propulsion mechanics, and computational optimization techniques.

Each feature is implemented as an independent but interconnected module, allowing the system to remain modular, extensible, and suitable for future research-level enhancements.

3.1 2D Motion Simulation

The 2D motion simulation forms the core of the rocket trajectory model. In real-world aerospace systems, rocket motion cannot be accurately represented in a single dimension because the trajectory depends on both vertical ascent and horizontal displacement.

In this model, the rocket's position is resolved into two perpendicular components:

- Horizontal motion (x-direction), governed by initial velocity, launch angle, and drag forces
- Vertical motion (y-direction), governed by thrust, gravity, and atmospheric resistance

The simulation updates the rocket's velocity and position iteratively using numerical integration techniques over small time steps. This allows the system to approximate continuous motion through discrete computations.

The resulting trajectory is typically parabolic during atmospheric flight. However, depending on velocity and altitude conditions, the motion may gradually transition into a near-orbital path when gravitational influence becomes weaker and drag forces diminish.

This module enables detailed analysis of:

- Trajectory shape evolution over time
- Maximum altitude (apogee) achieved
- Horizontal range and flight coverage
- Sensitivity of launch angle on performance

3.2 Atmospheric Model

The atmospheric model is one of the most critical components of the simulation, as it defines how environmental conditions change with altitude and directly influence aerodynamic forces acting on the rocket.

In Earth's atmosphere, air density decreases exponentially with altitude due to the reduction in atmospheric pressure and molecular concentration. This behavior is mathematically modeled as:

$$\rho(h) = \rho_0 e^{-h/H} \quad (10)$$

where:

- $\rho(h)$ represents air density at altitude h
- ρ_0 represents sea-level air density
- H is the atmospheric scale height, which defines how quickly density decreases with altitude

This exponential decay model ensures that the simulation accurately reflects real atmospheric behavior. As altitude increases, the air becomes progressively thinner, resulting in reduced aerodynamic drag forces acting on the rocket.

The drag force is directly dependent on air density, which means that atmospheric variation significantly affects the rocket's acceleration profile. Near the Earth's surface, high air density produces strong resistance, limiting acceleration. As the rocket ascends, reduced density leads to smoother motion and improved efficiency.

This model ensures:

- High aerodynamic resistance during launch phase
- Gradual reduction in drag with altitude
- Realistic transition from atmospheric to near-space conditions

3.3 Multistage Rocket System

The multistage rocket system is based on a fundamental aerospace engineering principle used in modern rocket design to maximize efficiency and payload capacity. The main idea behind multistaging is to reduce structural mass progressively during flight.

A rocket is divided into multiple stages, each consisting of its own fuel, engine, and structural components. Once a stage has consumed all its fuel, it becomes redundant weight and is detached from the main vehicle.

This staged separation significantly improves performance by increasing the thrust-to-weight ratio of the remaining rocket structure.

The mass reduction after each stage separation is represented mathematically as:

$$m_{k+1} = m_k - m_{stage,k} \quad (11)$$

where:

- m_k is the total mass before separation
- $m_{stage,k}$ is the mass of the discarded stage
- m_{k+1} is the remaining mass after separation

This reduction in mass leads to a direct improvement in acceleration, as the same thrust is now acting on a lighter system.

Key advantages of multistage architecture include:

- Elimination of inactive structural mass
- Increased velocity gain per stage

- Higher achievable altitude
- Improved fuel efficiency across the mission profile

In the simulation, stage separation is implemented as an event-driven mechanism triggered automatically based on fuel depletion conditions, ensuring realistic dynamic behavior.

3.4 AI Optimization (Starter Level)

The AI optimization module introduces an intelligent computational layer that enhances the performance of the rocket simulation by automatically tuning key input parameters. These parameters include launch angle, thrust magnitude, and fuel consumption strategy.

Instead of relying on manual selection, the system evaluates multiple parameter configurations and identifies the optimal set based on performance outcomes.

The optimization goal is to maximize flight efficiency while balancing fuel usage. This is expressed using the objective function:

$$J = \max(R, h) - \lambda E_{fuel} \quad (12)$$

where:

- R represents horizontal range
- h represents maximum altitude achieved
- E_{fuel} represents total fuel consumption
- λ is a weighting factor controlling fuel penalty

The optimization process is implemented using a simple heuristic or search-based strategy. The system evaluates different combinations of parameters and selects the configuration that yields the highest performance score.

This approach allows the system to:

- Automatically improve flight performance
- Reduce manual tuning of simulation parameters
- Explore optimal launch configurations efficiently

This module serves as a foundational step toward more advanced AI-based control systems such as genetic algorithms, swarm intelligence, or reinforcement learning methods, which can be integrated in future versions of the simulator.

4 Simple Vertical Rocket Motion Model

The simplest form of rocket flight simulation is the vertical motion model, in which the rocket is assumed to move only along the vertical axis (y-axis). This model serves as the foundation for understanding basic rocket dynamics before extending the simulation to more advanced two-dimensional or orbital motion systems.

In this simplified approach, the rocket experiences motion only in the upward direction, while gravity acts downward. The model primarily focuses on understanding how thrust, gravity, and changing mass influence rocket acceleration and altitude during flight.

The vertical rocket model is important because it introduces the fundamental physical principles required for all advanced rocket simulations. Even though this model ignores horizontal motion and aerodynamic complexity, it provides a clear understanding of force balance, fuel consumption, and numerical motion updates.

4.1 Basic Assumptions

To simplify the physical system, the following assumptions are considered in the initial model:

- The rocket moves only in the vertical direction.
- Horizontal motion is neglected.
- Gravity is assumed to remain constant near Earth's surface.
- Atmospheric drag is ignored in the simplest version.
- Thrust acts entirely in the upward direction.
- Rocket mass decreases continuously due to fuel burn.

These assumptions reduce the complexity of the system and allow the simulation to focus on the core mechanics of rocket propulsion.

4.2 Newton's Second Law for Vertical Motion

The motion of the rocket is governed by Newton's Second Law of Motion:

$$F = ma \tag{13}$$

For vertical rocket motion, the net force acting on the rocket is obtained by subtracting gravitational force from thrust force:

$$F_{net} = F_{thrust} - mg \tag{14}$$

Therefore, the acceleration of the rocket becomes:

$$a = \frac{F_{thrust} - mg}{m} \tag{15}$$

This equation shows that the rocket accelerates upward only when thrust exceeds the gravitational force.

4.3 Mass Variation Due to Fuel Burn

As the rocket engine burns fuel, the total mass of the rocket decreases continuously with time. This variation in mass is represented as:

$$m(t) = m_0 - \dot{m}t \quad (16)$$

where:

- m_0 = initial mass of the rocket
- $\dot{m}$ = fuel consumption rate
- t = time

This reduction in mass increases acceleration during flight because the same thrust acts on a lighter rocket.

4.4 Numerical Motion Update

The rocket motion is simulated numerically by updating acceleration, velocity, and position at small time intervals. At each simulation step:

- Acceleration is calculated from net force
- Velocity is updated using acceleration
- Position is updated using velocity
- Mass is updated based on fuel consumption

The numerical update equations are:

$$v(t + \Delta t) = v(t) + a(t)\Delta t \quad (17)$$

$$y(t + \Delta t) = y(t) + v(t)\Delta t \quad (18)$$

$$m(t + \Delta t) = m(t) - \dot{m}\Delta t \quad (19)$$

This iterative numerical method allows continuous rocket motion to be approximated using computational calculations.

4.5 Physical Interpretation

The vertical rocket model demonstrates the relationship between thrust, gravity, and mass variation. Initially, the rocket remains stationary if thrust is smaller than its weight. Once thrust exceeds gravitational force, the rocket begins to accelerate upward.

As fuel burns, the rocket becomes lighter, causing acceleration to increase progressively. This behavior explains why rockets accelerate rapidly during later stages of ascent.

Although simplified, this model forms the basis for advanced rocket simulations involving atmospheric drag, two-dimensional motion, multistage propulsion, and orbital mechanics.

4.6 Vertical Rocket Motion Diagram

Figure 1 illustrates the simplified vertical rocket motion model, including force balance, governing equations, numerical update process, and trajectory behavior.

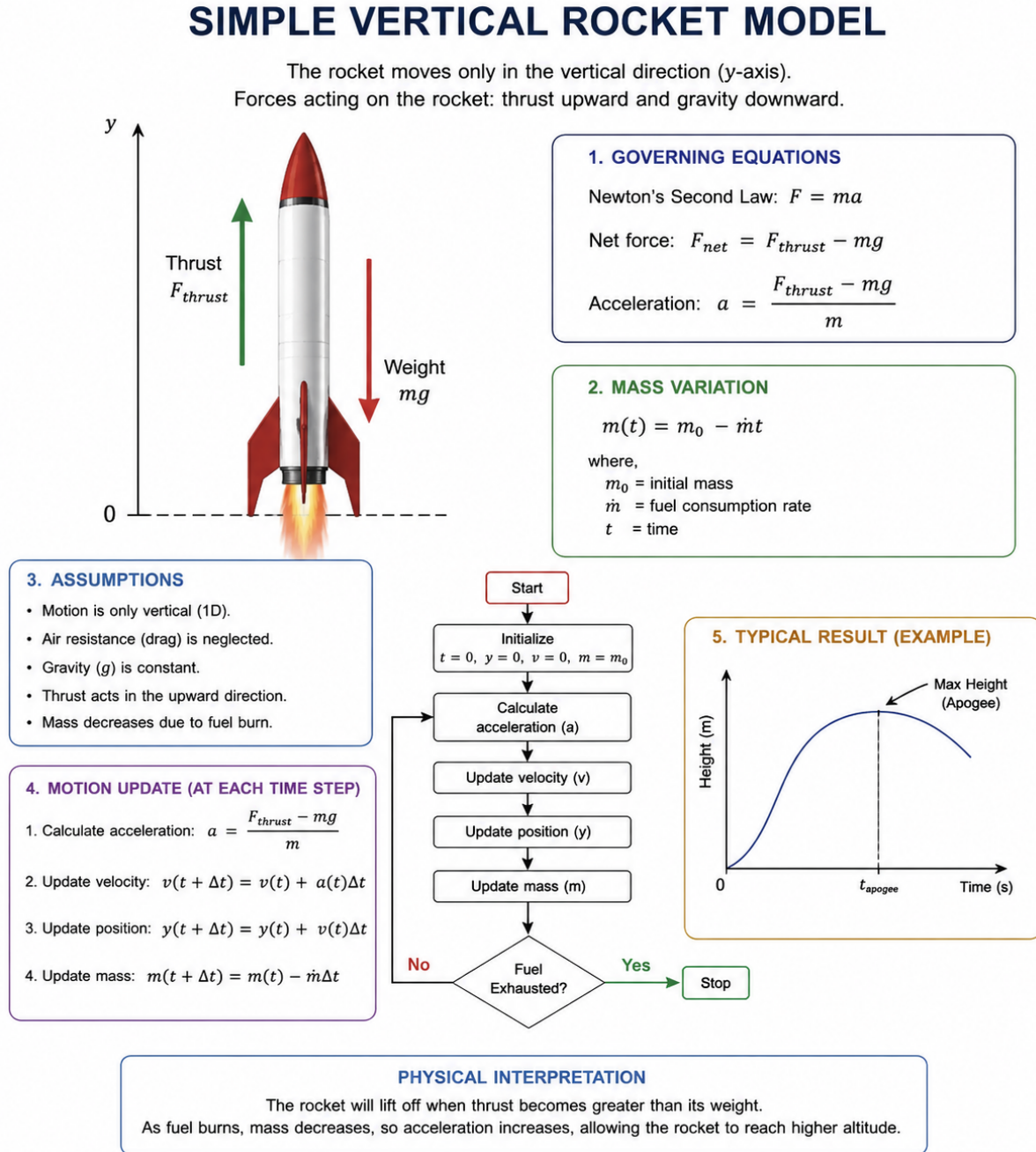


Figure 1: Simple Vertical Rocket Motion Model

5 Rocket Visualization and Advanced Simulation Models

This section presents the progressive development of the rocket flight simulator from a simple visual launch model to advanced artificial intelligence driven aerospace simulation systems. The simulator combines physics, numerical computation, graphical rendering, trajectory visualization, and AI-based optimization into a unified computational framework.

The development process begins with a graphical rocket visualization system using the Pygame library and gradually extends toward realistic atmospheric physics, data visualization, two-dimensional trajectory optimization, and finally three-dimensional AI-enhanced aerospace simulation.

The objective of these implementations is to create an educational and research-oriented rocket simulation platform capable of visualizing real physical processes involved in rocket propulsion and flight dynamics.

5.1 Rocket Visualization System

The first stage of the project focuses on creating a real-time graphical rocket visualization system using Python and the Pygame library. This module provides a visually interactive launch environment where the rocket ascends dynamically under the influence of thrust and gravity.

The visualization system includes:

- Rocket body rendering
- Engine flame animation
- Smoke particle effects
- Star-field background
- Launch pad visualization
- Real-time telemetry display

The rocket motion is governed by Newton's Second Law:

$$F = ma \tag{20}$$

The net force acting on the rocket is given by:

$$F_{net} = F_{thrust} - mg \tag{21}$$

As fuel burns continuously, the total rocket mass decreases with time, leading to dynamic acceleration changes during ascent.

The visualization module helps users understand how thrust, gravity, fuel consumption, and acceleration interact during rocket launch.

Python Implementation

```
1 # Rocket Visualization System
2 # Pygame Based Rocket Launch Simulator
3
4 import pygame
5 import sys
6 import random
7
8 pygame.init()
9
10 WIDTH = 800
11 HEIGHT = 900
12
13 screen = pygame.display.set_mode((WIDTH, HEIGHT))
14 pygame.display.set_caption("Rocket Flight Simulator")
15
16 clock = pygame.time.Clock()
17
18 WHITE = (255,255,255)
19 BLACK = (0,0,0)
20 DARK_BLUE = (5,5,30)
21 GRAY = (220,220,220)
22 RED = (220,30,30)
23 ORANGE = (255,140,0)
24
25 rocket_x = WIDTH//2
26 rocket_y = HEIGHT - 250
27
28 velocity = 0
29 g = 9.81
30
31 running = True
32
33 while running:
34
35     for event in pygame.event.get():
36         if event.type == pygame.QUIT:
37             running = False
38
39     velocity += 0.05
40     rocket_y -= velocity
41
42     screen.fill(DARK_BLUE)
43
44     pygame.draw.rect(
45         screen,
46         GRAY,
47         (rocket_x, rocket_y, 50, 150)
48     )
49
```

```

50     pygame.draw.polygon(
51         screen,
52         RED,
53         [
54             (rocket_x, rocket_y),
55             (rocket_x + 25, rocket_y - 60),
56             (rocket_x + 50, rocket_y)
57         ]
58     )
59
60     pygame.display.update()
61
62     pygame.quit()
63     sys.exit()

```

5.2 Realistic Rocket Physics Model

The next stage extends the visualization system into a realistic physics-based rocket simulator by incorporating aerodynamic drag, fuel burn mechanics, thrust generation, and dynamic mass reduction.

The drag force is calculated using:

$$F_d = \frac{1}{2} C_d \rho A v^2 \quad (22)$$

where:

- C_d = drag coefficient
- ρ = air density
- A = cross-sectional area
- v = velocity

The rocket mass changes continuously due to fuel consumption:

$$m(t) = m_{dry} + m_{fuel}(t) \quad (23)$$

The simulation computes the rocket motion numerically at small time intervals using discrete timestep integration.

Python Implementation

```

1  # Realistic Rocket Physics Simulator
2
3  import pygame
4  import math
5
6  pygame.init()
7
8  g = 9.81

```

```

9  rho = 1.225
10 Cd = 0.75
11 A = 0.3
12
13 thrust_force = 15000
14
15 dry_mass = 500
16 fuel_mass = 400
17
18 velocity = 0
19 altitude = 0
20
21 dt = 0.05
22
23 while True:
24
25     if fuel_mass > 0:
26         fuel_mass -= 2 * dt
27         thrust = thrust_force
28     else:
29         thrust = 0
30
31     mass = dry_mass + fuel_mass
32
33     gravity_force = mass * g
34
35     drag_force = (
36         0.5 * rho * Cd * A * velocity**2
37     )
38
39     net_force = (
40         thrust
41         - gravity_force
42         - drag_force
43     )
44
45     acceleration = net_force / mass
46
47     velocity += acceleration * dt
48     altitude += velocity * dt

```

5.3 Rocket Motion Visualization and Graph Analysis

To analyze rocket performance scientifically, visualization graphs are generated using the Matplotlib library. These plots help study the evolution of rocket motion during flight.

The simulator stores real-time data including:

- Altitude
- Velocity

- Fuel mass
- Simulation time

Three major graphs are generated:

1. Altitude vs Time
2. Velocity vs Time
3. Fuel Mass vs Time

These graphs provide valuable insight into rocket dynamics and fuel efficiency.

Altitude Graph

The altitude graph shows how the rocket gains height during powered ascent.

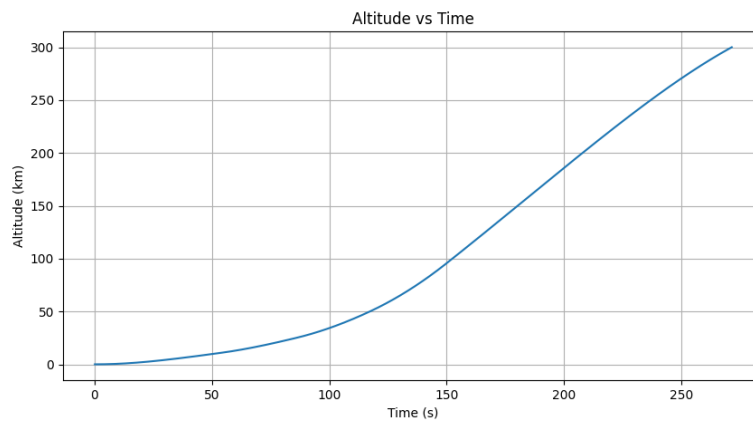


Figure 2: Rocket Altitude vs Time

Velocity Graph

The velocity graph illustrates acceleration behavior and velocity growth during thrust generation.

Fuel Mass Graph

The fuel graph demonstrates the continuous reduction of fuel mass during engine burn.

5.4 AI-Based 2D Rocket Optimization

The simulator is further extended into a two-dimensional rocket trajectory optimization framework using artificial intelligence techniques.

The motion now includes:

- Horizontal motion
- Vertical motion

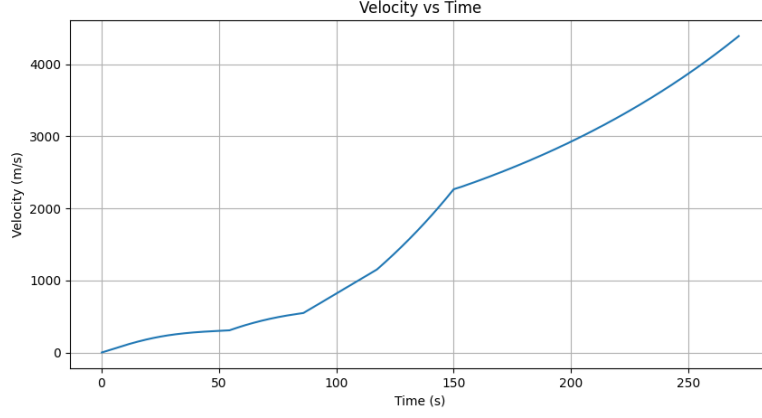


Figure 3: Rocket Velocity vs Time

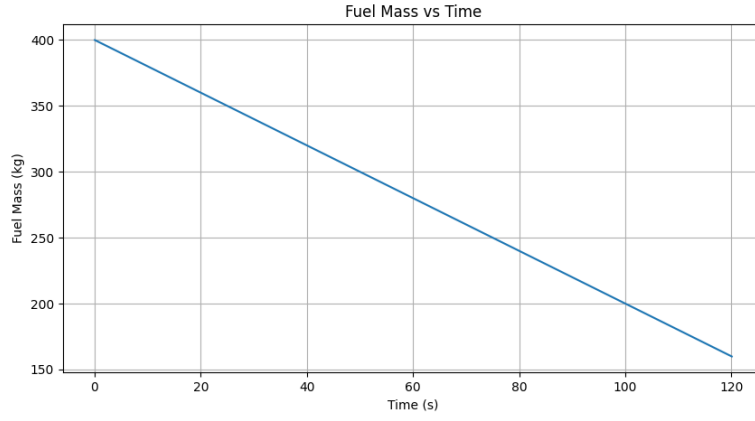


Figure 4: Fuel Mass vs Time

- Atmospheric density variation
- Two-stage rocket separation
- Drag force decomposition

Air density variation is modeled using:

$$\rho(h) = \rho_0 e^{-h/H} \quad (24)$$

where:

- ρ_0 = sea level air density
- h = altitude
- H = atmospheric scale height

The AI optimization system evaluates multiple launch angles and thrust combinations to maximize altitude performance.

A grid-search optimization algorithm is implemented to identify the best launch parameters automatically.

Features

- 2D trajectory simulation
- Multistage rocket physics
- Atmospheric drag
- AI parameter optimization
- Automatic trajectory evaluation

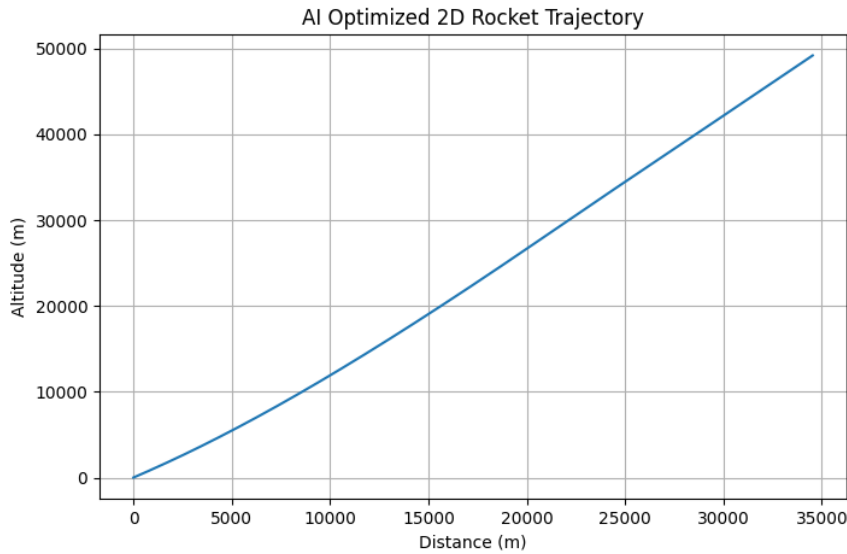


Figure 5: AI Optimized 2D trajectory

5.5 Advanced 3D AI-Based Aerospace Simulation

The final stage of the project introduces a three-dimensional aerospace simulation framework with advanced AI optimization capabilities.

The simulator incorporates:

- 3D rocket motion
- Wind shear modeling
- Central gravitational field
- Multistage propulsion
- Orbital trajectory estimation
- Genetic Algorithm optimization

The gravitational field is modeled using:

$$g = \frac{\mu}{r^2} \quad (25)$$

where:

- μ = Earth's gravitational parameter
- r = radial distance from Earth's center

A Genetic Algorithm is used to optimize:

- Launch angle
- Azimuth angle
- Stage thrust values

The AI system evolves multiple candidate trajectories and selects the highest-performing solution based on altitude and orbital conditions.

The final 3D trajectory is visualized using Matplotlib 3D plotting tools.

AI Optimized 3D Rocket Trajectory (Level 2 + 3)

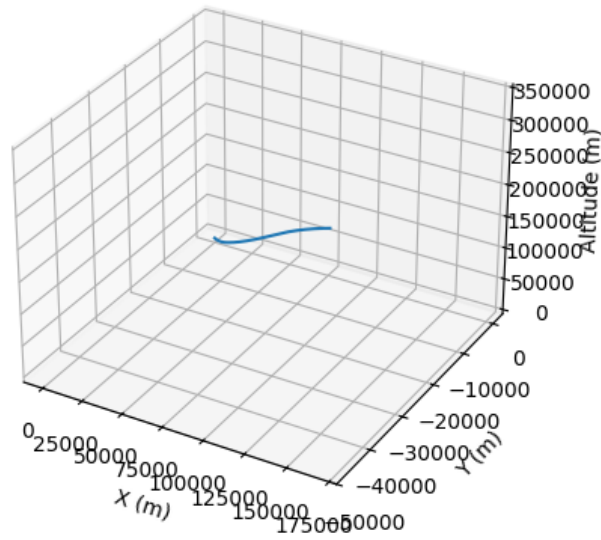


Figure 6: AI Optimized 3D Rocket Trajectory

6 Conclusion

This project successfully demonstrates the development of a comprehensive physics-based rocket flight simulator using Python. The simulator combines fundamental principles of classical mechanics with modern computational techniques to model realistic rocket behavior under different flight conditions. Throughout the project, important aerospace concepts such as gravitational variation, atmospheric drag, thrust generation, fuel consumption, and multistage rocket separation were mathematically modeled and computationally implemented to create a dynamic and extensible simulation environment.

The project initially began with a simple one-dimensional vertical rocket motion model in order to understand the fundamental relationship between force, mass, acceleration, and fuel burn. Using Newton's Second Law of Motion, the simulator was gradually extended into more advanced levels incorporating atmospheric resistance and varying rocket mass during propulsion. This provided a strong foundation for understanding how real rockets behave during launch and ascent phases.

Further improvements introduced two-dimensional rocket trajectory simulation, allowing both vertical and horizontal motion to be analyzed simultaneously. This enhancement enabled the visualization of realistic flight paths, trajectory curvature, and long-range motion behavior. The implementation of atmospheric density variation using exponential atmospheric models improved the realism of aerodynamic drag calculations and demonstrated the significant effect of air resistance on rocket performance.

One of the major achievements of this project is the successful implementation of multistage rocket dynamics. The simulator accurately models stage-wise fuel depletion and automatic separation mechanisms, thereby reducing total mass during flight and improving thrust efficiency. This reflects the working principle used in real-world launch vehicles where stage separation plays a critical role in achieving higher altitude and orbital capability.

The project also integrates advanced visualization systems using Pygame and Matplotlib to provide real-time graphical representation of rocket motion and simulation parameters. Parameters such as altitude, velocity, acceleration, and fuel mass were continuously monitored and visualized through animated flight systems and performance graphs. These visual components significantly improve interpretability and educational understanding of rocket dynamics.

In addition to classical simulation methods, the project introduces an initial Artificial Intelligence based optimization framework for trajectory improvement. AI-based search techniques were used to optimize launch angle and thrust parameters in order to maximize altitude and improve flight efficiency. The final stages of the project further expanded into research-level three-dimensional simulation involving atmospheric wind effects, aerodynamic drag in 3D space, and genetic algorithm based optimization methods. These features demonstrate the potential for extending the simulator into advanced aerospace research applications.

Overall, the project successfully bridges theoretical physics, numerical computation, aerospace engineering concepts, and intelligent optimization into a single integrated simulation platform. The developed system not only serves as an educational tool for understanding rocket propulsion and flight mechanics, but also establishes a strong computational foundation for future research in aerospace simulation, orbital mechanics, AI-driven trajectory optimization, and autonomous space systems.

References

1. Sutton, G. P., and Biblarz, O., *Rocket Propulsion Elements*, 8th Edition, John Wiley & Sons, 2010.
2. Anderson, J. D., *Introduction to Flight*, McGraw-Hill Education, 2015.
3. Vallado, D. A., *Fundamentals of Astrodynamics and Applications*, 4th Edition, Springer, 2013.
4. White, F. M., *Fluid Mechanics*, 8th Edition, McGraw-Hill, 2016.
5. OpenAI, *Artificial Intelligence Optimization Methods for Simulation Systems*, 2025.
6. NumPy Developers, *NumPy Documentation*, Available at: <https://numpy.org/>
7. Matplotlib Development Team, *Matplotlib Visualization Library*, Available at: <https://matplotlib.org/>
8. Pygame Community, *Pygame Documentation*, Available at: <https://www.pygame.org/docs/>
9. Python Software Foundation, *Python Programming Language Documentation*, Available at: <https://www.python.org/>
10. NASA Glenn Research Center, *Beginner's Guide to Rockets*, Available at: <https://www.grc.nasa.gov/>
11. European Space Agency (ESA), *Rocket and Spacecraft Dynamics*, Available at: <https://www.esa.int/>
12. Curtis, H. D., *Orbital Mechanics for Engineering Students*, Elsevier Aerospace Engineering Series, 2014.
13. Wertz, J. R., and Larson, W. J., *Space Mission Analysis and Design*, Microcosm Press, 1999.
14. McKinney, W., *Python for Data Analysis*, O'Reilly Media, 2022.
15. Press, W. H., et al., *Numerical Recipes: The Art of Scientific Computing*, Cambridge University Press, 2007.